Hvordan påvirker bruk av teknologiske tester effektiviteten og kodekvaliteten i programvareutviklingsprosessen?[1]

Innledning

I vårt prosjekt satte vi fokus på å benytte oss av en metodikk kalt «Smidige testkvadranter». Ifølge Lisa Crispin og Janet Gregory er denne kvadranten et viktig fundament for å ha et smidig team som sørger for raske tilbakemeldinger ved hjelp av automatiske tester. Denne kvadranten deler testingen inn i fire kvadranter som sammen sikrer at testingen dekker alle aspekter av produktet, fra tekniske detaljer til forretningskrav (Crispin & Gregory, 2008, s. 97). Innad i denne kvadranten setter kvadrant 1 og 2 fokus på de delene av kvadrantene som tar for seg tester som støtter teamet. Kvadrant 1 setter videre fokus på tekniske tester, mens kvadrant 2 tar mer for seg hvordan produktet funker fra en brukers perspektiv. I denne teksten vil vi utforske hvordan bruk av teknologiske tester støtter teamet i programvareutviklingsprosessen, med fokus på testdrevet utvikling (TDD) og integrasjon av automatiserte tester i kontinuerlig integrasjon (CI). Jeg vil undersøke hvordan disse metodene påvirker effektiviteten og kodekvaliteten, samt diskutere fordeler, utfordringer og beste praksiser basert på egne erfaringer og teoretiske perspektiver.

Testdrevet utvikling

Testdrevet utvikling er en praksis der tester skrives før koden gjennomføres. Denne praksisen betyr at utviklerne først definerer testkravene, deretter implementerer koden som oppfyller disse kravene og til slutt kjører testene for å validere koden (Kniberg, 2015, s. 105-106). Dette gir utviklerne et klart mål å jobbe mot og sikrer at koden er testbar og robust fra begynnelsen av utviklingsprosessen. Fokuset på å skrive unittester før selve koden krevet også en felles forståelse for produktets krav og funksjonalitet som ønskes. Denne felles forståelsen skapte et gruppemiljø støttet av åpenhet og tilbakemeldinger. Ved å tvinge utviklingsteam til å tenke gjennom kravene til koden før implementering, oppmuntret TDD til å skrive strukturerte kodeløsninger. TDD gir også kontinuerlig tilbakemelding og validering av koden, som fører til

[1] Erklæring om bruk av kunstig intelligens: Google Gemini Advanced brukt for å finne synonymer til ord.

høyere tillit til koden og redusert forekomst av feil og defekter (Nagappan et al., 2008). Denne reduserte forekomsten av feil og defekter vises å være på hele 40 til 90%, denne store forbedring kommer også på bekostning av en økning på mellom 15 og 35% økt oppstarts tid (Nagappan et al., 2008). Dette fører til en mer vedlikeholdbar og fleksibel kodebase som er enklere å utvide og endre over tid dersom det endrer på prosjekt strukturen. Disse fordelene ble i stor grad merket i prosjektarbeidet vårt, prosjektet vårt brukte ikke TDD i så stor grad som vi senere ønsket og dette ba på flere utfordringer.

Bruk av testdrevet utvikling og utfordringer

Nye og mer uerfarne utviklere kan oppleve en bratt læringskurve når de først begynner å praktisere TDD, og det kan være vanskelig å motstå fristelsen til å hoppe over testfasen og gå direkte til koding. Denne feilen kan adresseres ved å ha en person som har primærfokus på tester (Kniberg, 2015. s. 116). I gruppen hadde vi ingen som følte seg komfortabel med å ta på seg dette ansvaret, så det ble dermed fordelt på alle gruppemedlemmer. Dette opplevde jeg som ikke alltid like lett, ihvertfall når du tar for deg en ny type teknologi du ikke har mye tidligere erfaring med, denne følelsen var felles for store deler av gruppen som førte til inkonsistent bruk av TDD. Erfaringen med inkonsekvent bruk av TDD i gruppen fikk også fram hvor hensiktsmessig bruken av TDD faktisk var. De områdene hvor TDD ikke var brukt, skapte i stor grad problemer ved videre utvikling som trolig ikke ville fantes hvis vi var mer konsistente med bruken gjennom hele prosjektet. Derimot hvor vi benyttet oss av TDD, ble det mindre problemer senere i utviklingen som vi måtte tilbake å endre på. Mangelen på gjennomføring av TDD ble ikke merket i starten av prosjektet, men ettersom produktet blir mer og mer komplekst ble mangelen på tester mer åpenbar. Senere i prosessen oppdaget vi også et problem der kode som ikke tok i bruk TDD var vanskeligere å innføre TDD på, hvilket også er tatt opp i pensum (Kniberg, 2015, s. 107). Den negative erfaringen gruppen opplevde ved å ikke ta i bruk TDD motiverte oss til å utvikle en fremtidig handlingsplan hvor dette var mer sentralt. Nå som jeg også er mer erfaren med teknologi brukt har jeg tro på at utføringen av TDD vil være mye mer effektivt samt by på alle positive sider som nevnt over.

Integrasjon av automatiserte tester

Integrasjonen av automatiserte tester i kontinuerlig integrasjon (CI) muliggjør rask validering av kodeendringer og tidlig oppdagelse av feil (Kniberg, 2015, s. 109). Dette bidrar til å opprettholde høy kvalitet på produktet og smidige endringer. Dette reduserer risikoen for å introdusere feil og defekter i koden (Athanasiou et al., 2014). Automatiserte tester gir også muligheten til å oppdage feil tidlig i utviklingsprosessen, noe som resulterer i raskere feilretting og smidige endringer (Bomfim & Santos, 2017). Disse testene ble kjørt automatisk hver gang det ble gjort endringer, hvilket ga teamet muligheten å raskt validere at de leverte funksjonene oppfyller forventningene, noe som bidro til å opprettholde høy kvalitet på produktet som helhet. Automatiserte tester integrert i CI-pipelinen inkluderte enhetstester, integrasjonstester og akseptansetester. Feilene funnet ved hjelp av denne mekanismen kunne være alt fra små estetiske problemer til større feil som sørget for at prosjektet ikke vil kjøre korrekt lenger. Dette bidro til smidige endringer for å kunne fikse problemene før vi fortsatte å integrere flere endringer. Signifikansen av bruken av CI-pipeline kommer fram når vi ser at cirka. 40% av våre «pipelines» som ble kjørt hadde problemer som måtte justeres. Som sagt var mange av disse feilene, mindre estetiske problemer som ikke ga store problemer, men disse sørget også for at kodekvaliteten skulle være god over hele prosjektet. En stor fordel jeg merket av å investere tid i en god pipeline for prosjektet var hvor lite som krevdes for å benytte seg av den, men hvor mye den ga tilbake i form at spart tid og innsats.

Gruppens utvikling og anbefalinger

Fra start til slutt har det vært en stor endring i både mitt og gruppens syn på implementering av tester. Til å starte med så vi selvfølgelig fordelene ved å implementere tester, men var redd et for stort fokus på dette ville påvirke tiden til utvikling av prosjektet. I løpet av de to sprintene reagerte teamet positivt på implementeringen av tester, men vi møtte utfordringer med å oppnå det ønskede nivået av testdekning. Dette førte til diskusjoner om hvordan vi kunne forbedre testingen og tilpasse arbeidsmetodene våre for å integrere testing mer sømløst i utviklingsprosessen. Desto lenger inn i utviklingsprosessen vi kom jo flere positive aspekter så vi av de testene vi hadde, og negative aspekter av de vi ikke hadde. Dette kom fram i siste retrospektiv hvor et stort ønske var å kontinuerlig legge til flere tester, samt å kunne sette et mål

på 80% testdekning til neste sprint. Ut ifra min erfaring med prosjektet er det flere tips jeg har lært som jeg vil benytte til videre prosjekt arbeid, hvilket vil også være nyttig til nye grupper i liknende prosjektarbeid. Start TDD og testing tidlig i prosjektet og fortsett kontinuerlig gjennom hele utviklingsprosessen for å oppnå best mulig resultat (Nagappan et al., 2008). Selv om å integrere tester virker lite lukrativt til å begynne med vil ett fokus på tester fra starten av lønne seg veldig i lengden. Ettersom produsering av testing vil sikre en god grunnstruktur og å få teamet til å ha en felles tolkning av hvordan oppgaver og problemer skal løses. Konsentrasjon på en god CI-pipeline vil lønne seg i stor grad fra starten av til å opprettholde fin kode med høy kvalitetsstandard. Dette var også nyttig for å ha et sikkerhetsnett for de som kanskje ikke følte seg fult så komfortabel med å gjøre store endringer.

Konklusjon

Ut ifra teori og egne erfaringer kan jeg konkludere med at implementeringen av teknologiske tester har en betydelig positiv innvirkning på effektiviteten og kodekvaliteten. Testdrevet utvikling har vist seg å være en verdifull praksis for å sikre høy kvalitet på koden, redusere feil og defekter, samt oppmuntre til en felles forståelse blant teammedlemmene. Til tross for den bratte læringskurven og utfordringene med å opprettholde konsistens, har TDD vist seg å være en verdifull tilnærming som gir betydelige fordeler på lang sikt. Integrasjonen av automatiserte tester i kontinuerlig integrasjon har også vist seg å være avgjørende. Gjennom prosessen har gruppen erfart positive effekter av disse prosessene samt et savn på å ikke benyttet dette i større grad. Mens det var utfordringer med å oppnå ønsket testdekning og opprettholde konsistens, ble verdien av tester stadig mer åpenbar ettersom prosjektet gikk fremover. Erfaringene har ført til en økt forståelse for viktigheten av å starte tidlig med testing og å opprettholde kontinuerlig innsats gjennom hele utviklingsprosessen. Jeg har dermed stor tro på at å benytte seg av testdrevet utvikling fra starten av vil oppnå høyere samlet effektivitet og kunne levere høykvalitetsprogramvare til sluttbrukerne.

Referanseliste

Kniberg, H. (2015). Scrum and XP from the Trenches - 2nd Edition. C4Media. ISBN: 978-1-392-22427-8.

Crispin, L., & Gregory, J. (2008). Agile Testing: A Practical Guide for Testers and Agile Teams. Addison-Wesley Professional. ISBN: 9780321534460.

Bomfim, M. M., & Santos, V. B. D. (2017). Strategies for reducing technical debt in agile teams. In *Communications in computer and information science* (pp. 60–71). https://doi.org/10.1007/978-3-319-55907-0_6

Athanasiou, D., Nugroho, A., Visser, J., & Zaidman, A. (2014). Test code quality and its relation to issue handling performance. *IEEE Transactions on Software Engineering*, *40*(11), 1100–1125. https://doi.org/10.1109/tse.2014.2342227

Nagappan, N., Maximilien, E. M., Bhat, T., & Williams, L. (2008). Realizing quality improvement through test driven development: results and experiences of four industrial teams. *Empirical Software Engineering (Print)*, *13*(3), 289–302. https://doi.org/10.1007/s10664-008-9062-z